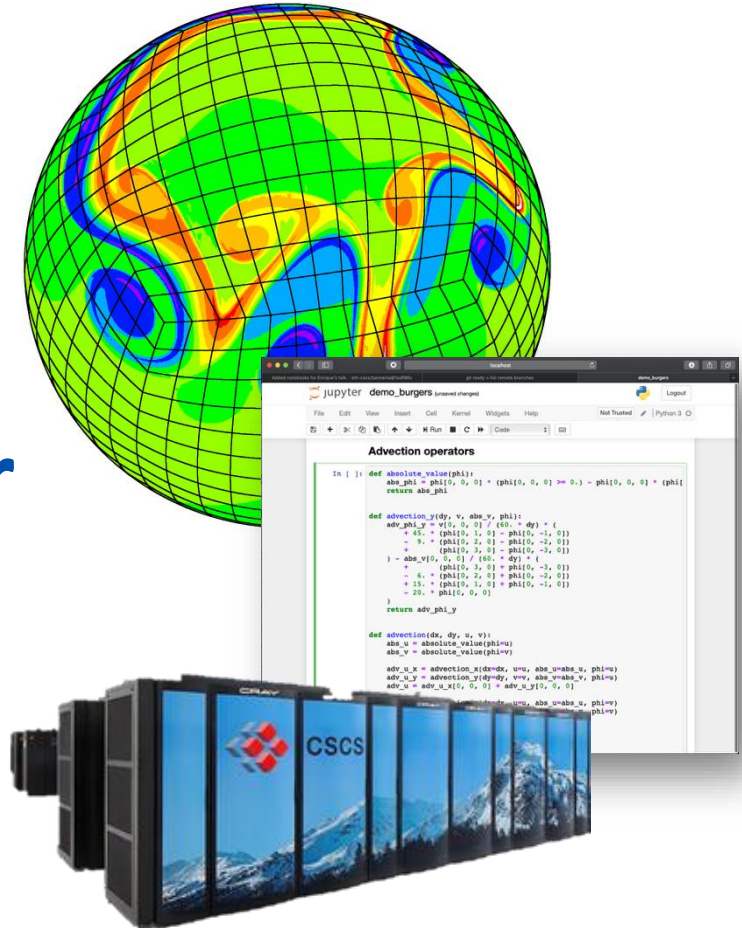


High Performance Computing for Weather and Climate (HPC4WC)

Content: Introduction
Lecturer: Oliver Fuhrer
Block course 701-1270-00L
Summer 2026

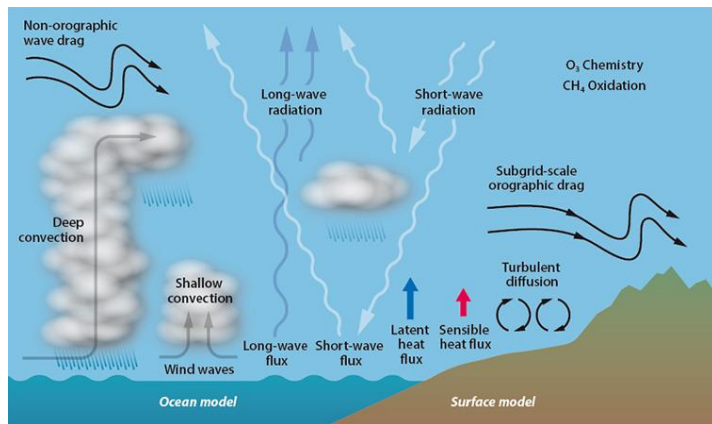


Learning goals

- Understand why weather and climate simulation requires high-performance computing (HPC)
- Understand why stencil computations are a key algorithmic motif of weather and climate models
- Understand a simple stencil program:
higher-order monotonic diffusion (Xue 2000, MWR)

Modeling the Earth system

Physical understanding



Governing equations
(e.g. atmosphere)

$$\frac{d}{dt} \mathbf{v} = -2\boldsymbol{\Omega} \times \mathbf{v} - \frac{1}{\rho} \nabla_3 p + \mathbf{g} + \mathbf{F}$$

Conservation of momentum
(Navier-Stokes)

$$C_v \frac{d}{dt} (\rho q) + p \frac{d}{dt} \left(\frac{1}{\rho} \right) = J$$

Conservation of energy
(1st Law of Thermodynamics)

$$\frac{\partial}{\partial t} (\rho) = -\nabla_3 \cdot (\rho \mathbf{v})$$

Conservation of air mass

$$\frac{\partial}{\partial t} = -\nabla_3 \cdot (\rho \mathbf{v} q) + \rho (E - C)$$

Continuity of water vapor mass

$$p = \rho RT$$

Equation of state
(Ideal gas law)

Variables: $\{\mathbf{v}, p, T, \rho, q\}$

Grids on a sphere



Tetrahedron



Hexahedron



Octahedron

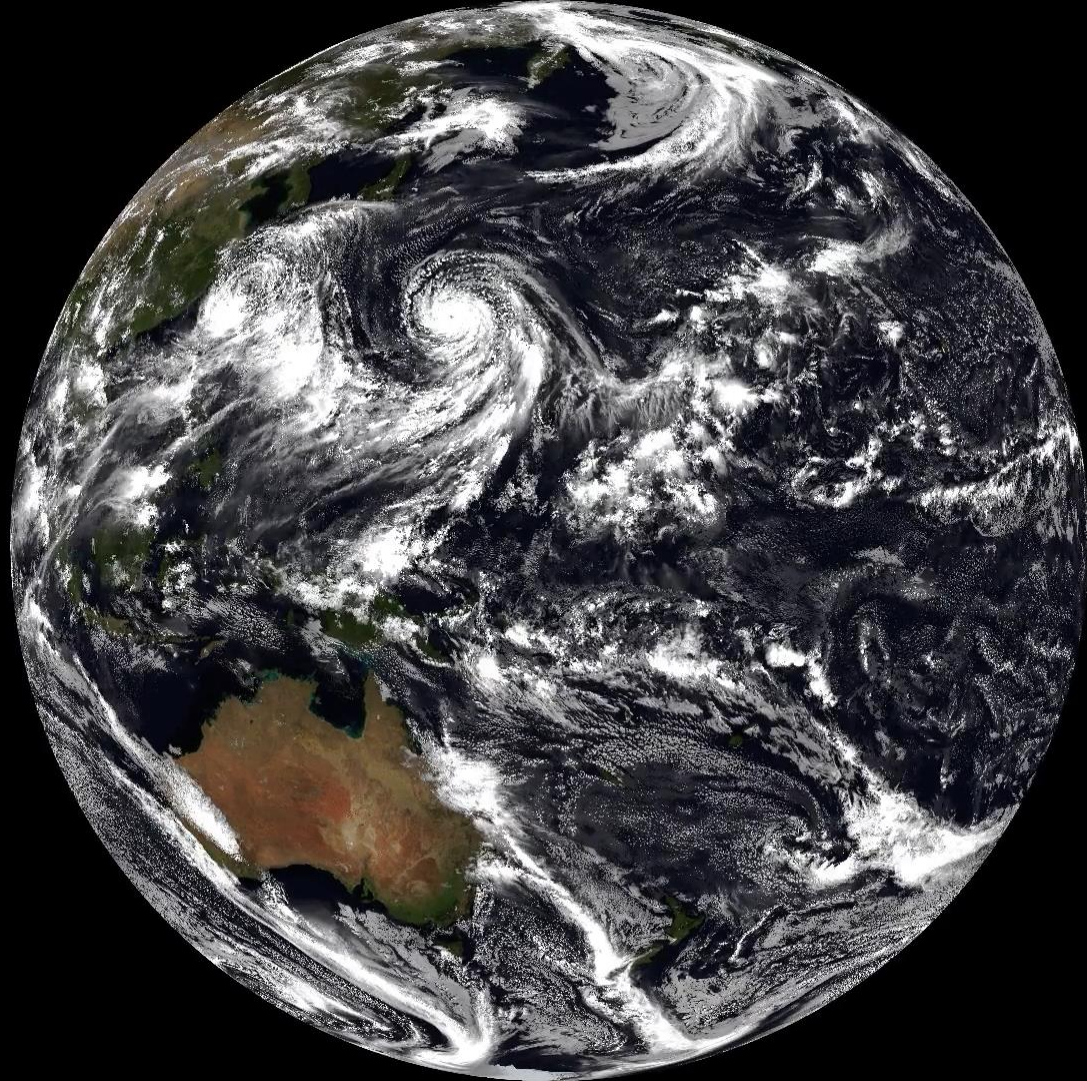


Dodecahedron



Icosahedron





$6 \times 3072^2 \times 80$

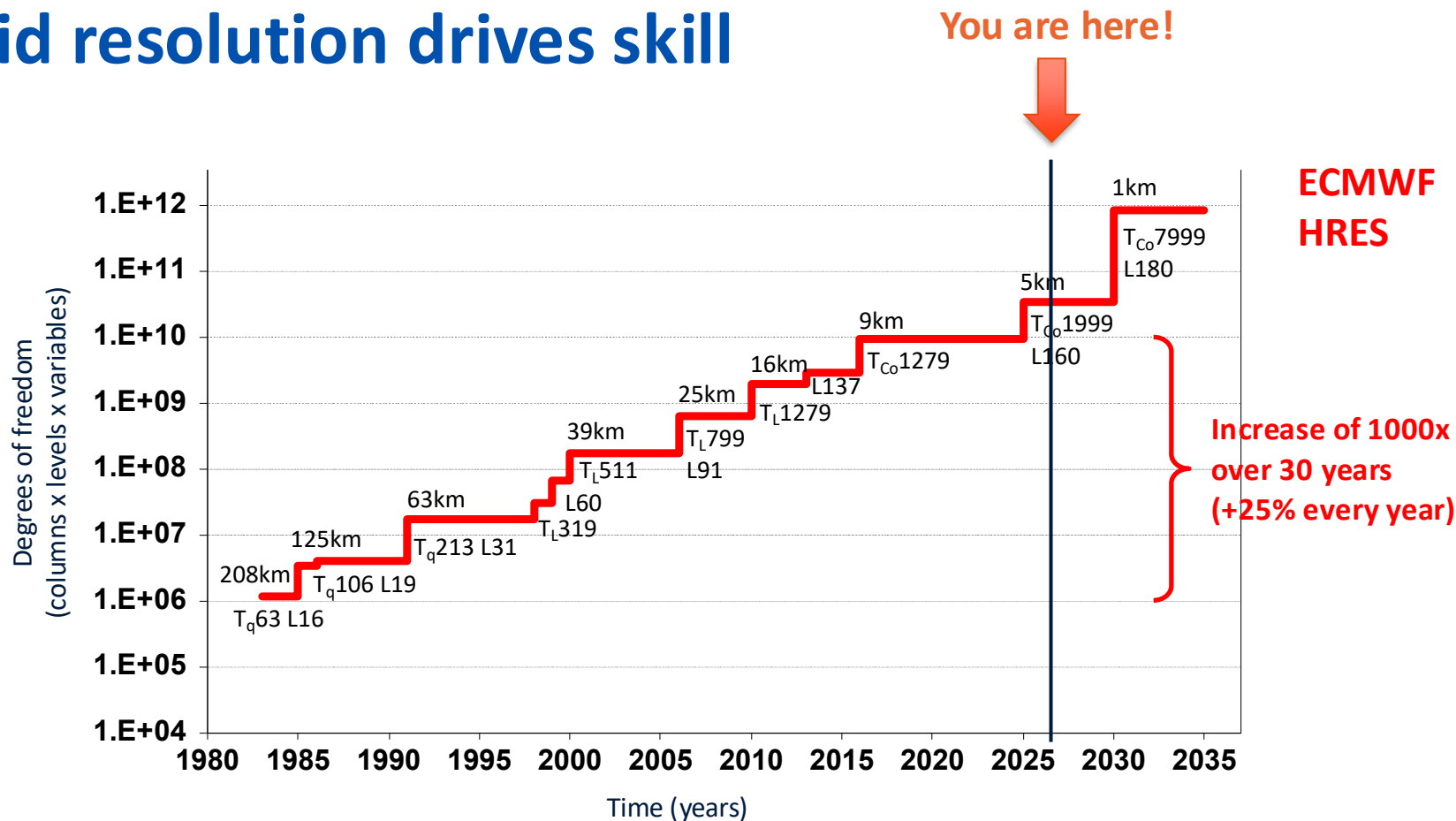
2016-08-11 18:00Z
258 Forecast Hours
FV3 3km

Forecast skill over time

You are here!

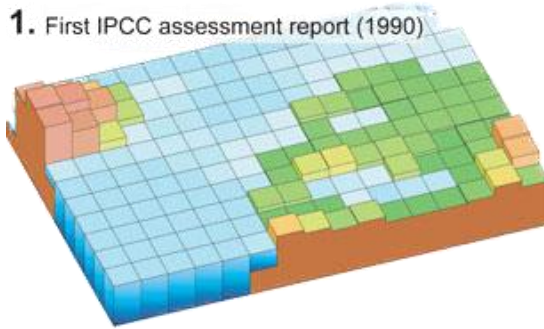


Grid resolution drives skill

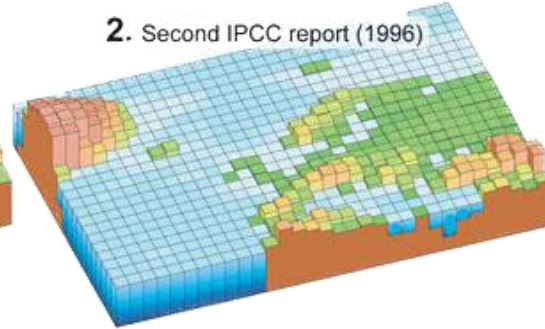


Grid resolution in the IPCC

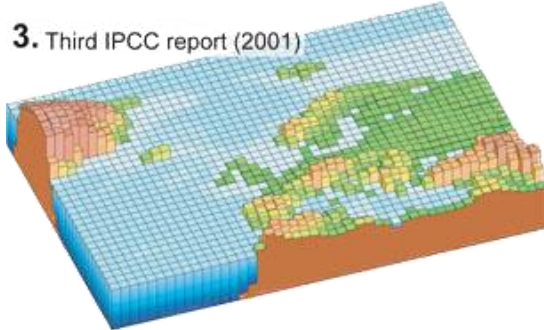
1. First IPCC assessment report (1990)



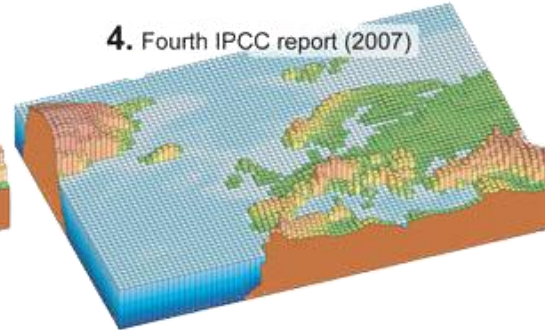
2. Second IPCC report (1996)



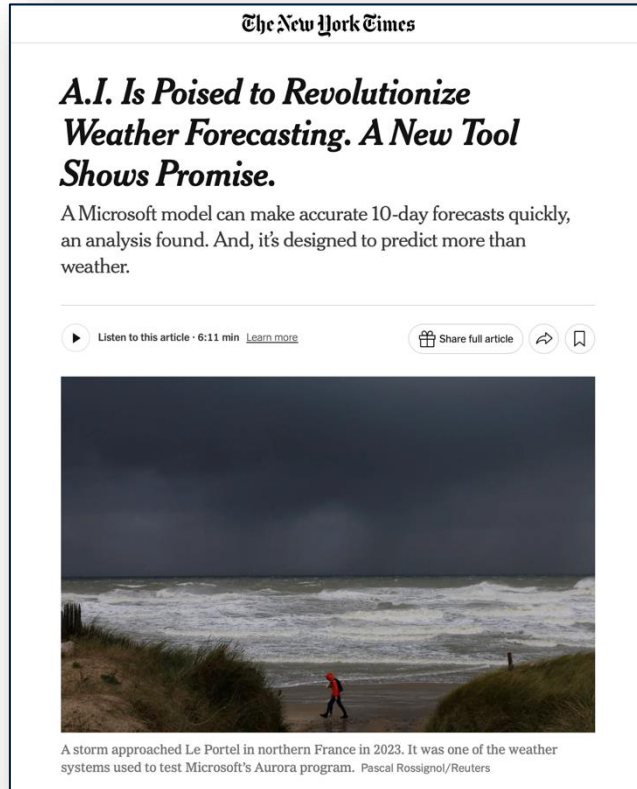
3. Third IPCC report (2001)



4. Fourth IPCC report (2007)

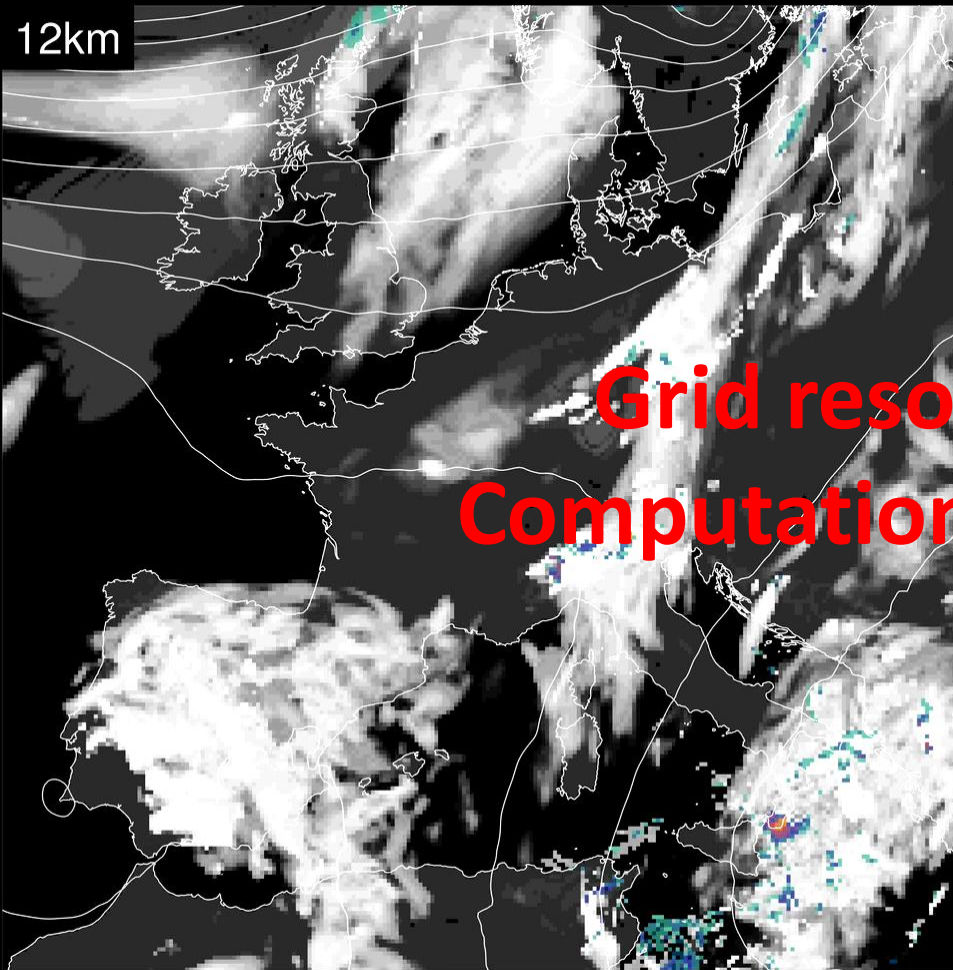


What about AI/ML?

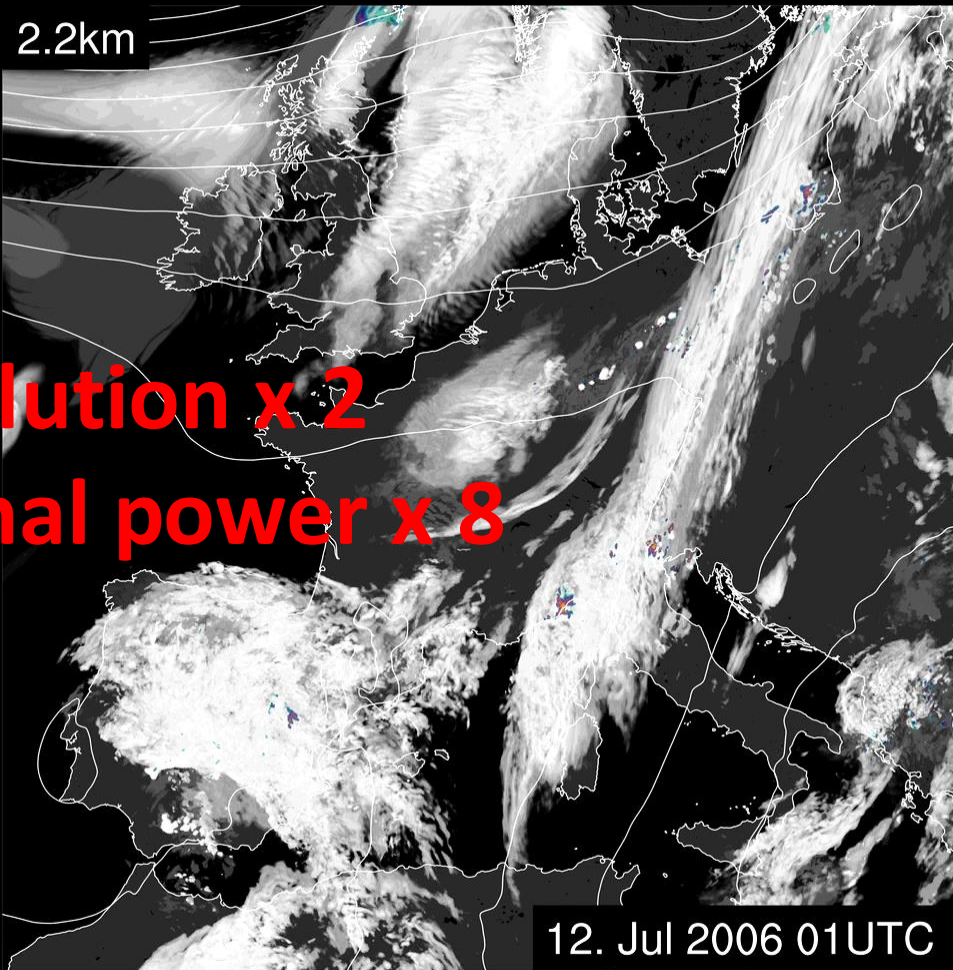


- On-going revolution with data-driven models making rapid progress
- (Almost) all data-driven models currently rely on datasets which have been generated by weather and climate models running on supercomputers

12km



2.2km

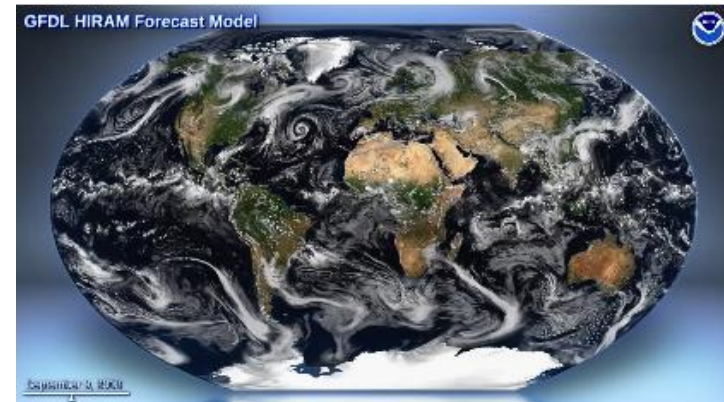
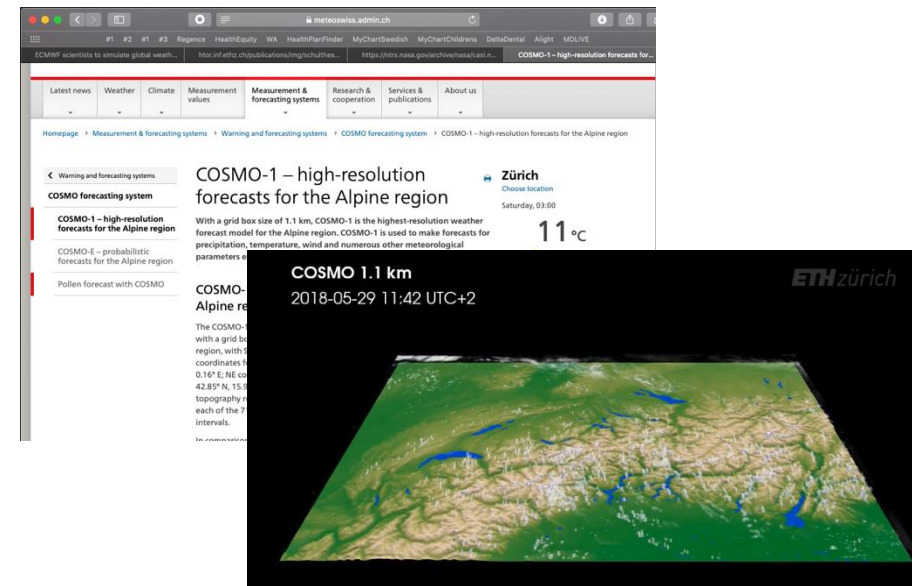


Grid resolution x 2
Computational power x 8

12. Jul 2006 01UTC

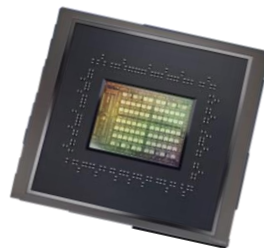
Example applications

- Swiss national weather forecast ([ICON-CH1-EPS](#))
 - $1.15 \times 10^6 \times 80$ gridpoints
 - $\Delta x = 1.05 \text{ km}$
 - $\Delta t = 10 \text{ s}$
 - 11 members
 - 80 x faster than real time (0.2 SYPD)
- Global coupled climate model with 1.25 km resolution ([ICON Sapphire](#))
 - $3.26 \times 10^8 \times 80$ gridpoints
 - $\Delta x = 1.25 \text{ km}$
 - $\Delta t = 10 \text{ s}$
 - 1000 x faster than real time (3 SYPD)



Why HPC?

- Swiss national weather forecast ([ICON-CH1-EPS](#))
 $1.15 \times 10^6 \times 80$ gridpoints
 $\Delta x = 1.05 \text{ km}$
 $\Delta t = 10 \text{ s}$
11 members
80 x faster than real time
- Global coupled climate model with 1.25 km resolution ([ICON Sapphire](#))
 $3.26 \times 10^8 \times 80$ gridpoints
 $\Delta x = 1.25 \text{ km}$
 $\Delta t = 10 \text{ s}$
~~1000 x faster than real time~~
Currently only 100 x feasible!



NVIDIA Grace CPU
@3.44 Ghz, 250 W

~ 1'000 CPUs

~ 250 kW (= 320 households)

~ 500'000 CPUs

~ 125 MW (approx. 10% of Gösgen)



~ 130 CPUs
(only sufficient for one ICON-CH1-EPS member)

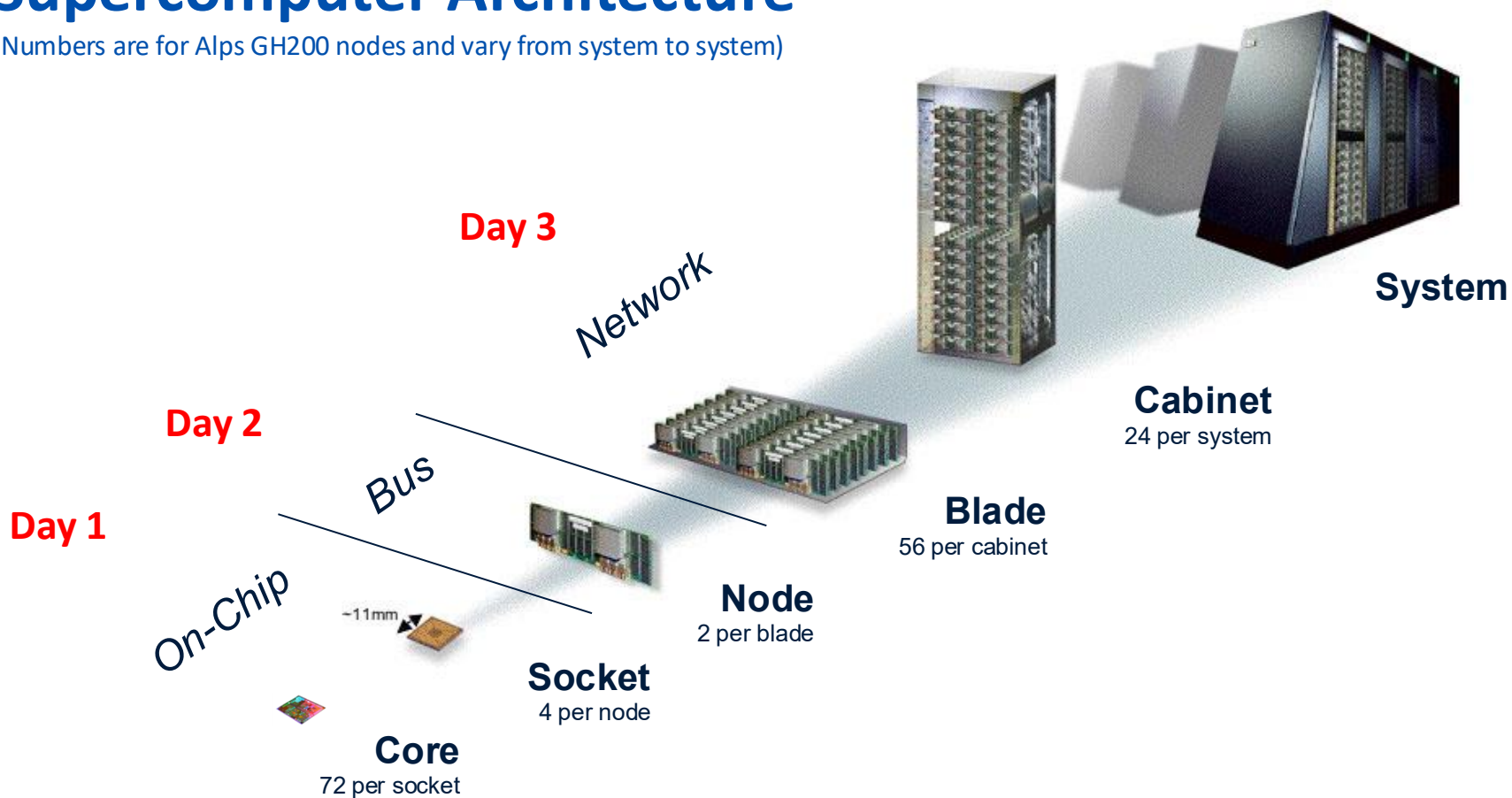


14'000 CPUs

(~ 15 x more dense!)

Supercomputer Architecture

(Numbers are for Alps GH200 nodes and vary from system to system)



“ *High Performance Computing* is the practice of aggregating computing power in a way that delivers much higher performance than one could get out of a typical desktop computer or workstation in order to solve large problems in science, engineering, or business. ”

Lab Exercises

01-roofline-model.ipynb

- Learn about performance metrics and how to compute them.
- Learn about arithmetic intensity and performance limiters.

*Solve notebooks without Bonus questions first.
If you're done, attack the Bonus questions.*

02-stencil-program.ipynb

- Determine arithmetic intensity of a stencil program.
- Apply a performance profiling tool to gain insight into performance.
- Show limitations of the von Neumann model for understanding performance.

03-caches-data-locality.ipynb

- Determine arithmetic intensity of a stencil program.
- Apply a performance profiling tool to gain insight into performance.
- Show limitations of the von Neumann model for understanding performance.

Learning goals

- Understand why weather and climate simulation requires high-performance computing (HPC)
- Understand why stencil computations are a key algorithmic motif of weather and climate models
- Understand a simple stencil program:
higher-order monotonic diffusion (Xue 2000, MWR)

Higher-order diffusion (Xue 2000, MWR)

- Atmospheric and ocean models often need some form of numerical filtering to control small-scale noise.
- [Xue 2000 \(Monthly Weather Review\)](#) published a class of higher-order monotonic filters that are frequently used.
- We will focus on the 4th-order non-monotonic diffusion for this course.



Governing equations

High-order monotonic diffusion
(see paper)

Simplifications (see paper)

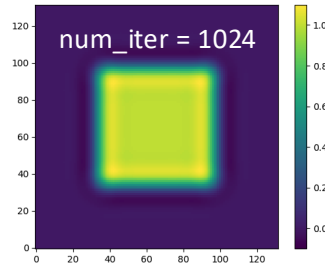
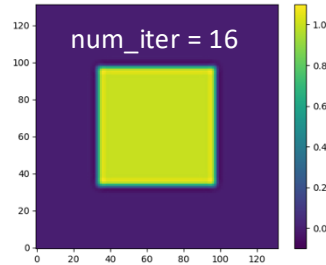
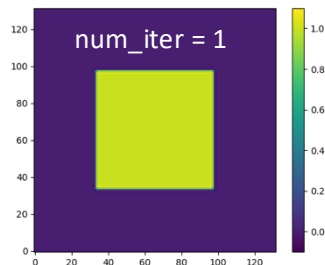
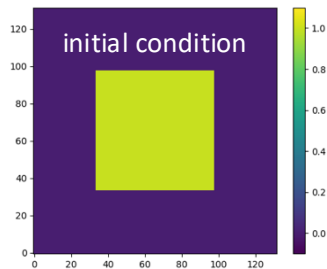
- ignore other processes ($S=0$)
- 4th-order ($n=4$)
- without limiter

$$\frac{\partial \phi}{\partial t} = S + (-1)^{n/2+1} \alpha_n \nabla^n \phi, \quad (1)$$

$$S = 0 \quad n = 4 \quad \text{horizontal}$$

$$\begin{aligned} \frac{\partial \phi}{\partial t} &= -\alpha_4 \nabla_h^4 \phi \\ &= -\alpha_4 \Delta_h^2 \phi \quad \Delta_h = \nabla_h^2 \end{aligned}$$

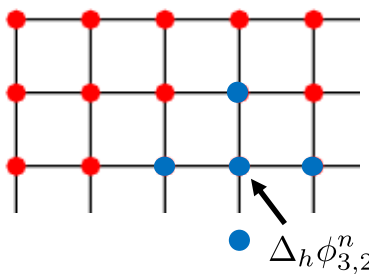
$$\frac{\partial \phi}{\partial t} = -\alpha_4 \Delta_h (\Delta_h \phi)$$



Discretization: Finite difference method (FDM)

Computational grid

Store values at gridpoints (i,j)



$$x_{i,j} = i \Delta x \quad y_{i,j} = j \Delta y$$

$$t^n = n \Delta t$$

$$\phi_{i,j}^n = \phi(x_{i,j}, y_{i,j}, t^n)$$

Spatial discretization

2nd-order centered

$$\Delta_h \phi_{i,j}^n \approx (-4 \phi_{i,j}^n + \phi_{i-1,j}^n + \phi_{i+1,j}^n + \phi_{i,j-1}^n + \phi_{i,j+1}^n) / \Delta x^2$$

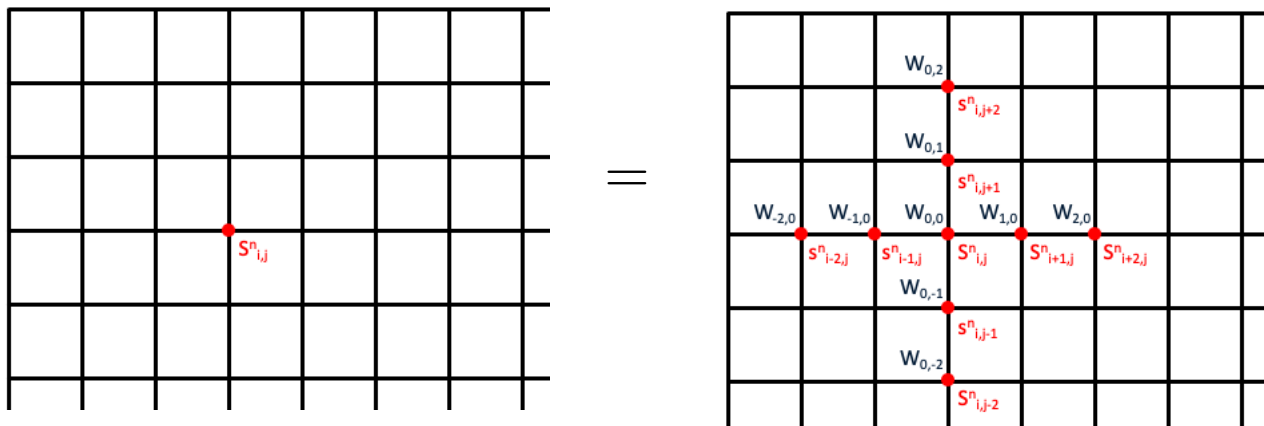
Time discretization

1st-order forward (Euler)

$$\partial_t \phi_{i,j}^n \approx (\phi_{i,j}^{n+1} - \phi_{i,j}^n) / \Delta t$$

Stencil computation (5x5, sparse)

$$s_{i,j}^{n+1} = \sum_{i_{\text{rel}}, j_{\text{rel}}} w_{i_{\text{rel}}, j_{\text{rel}}} s_{i+i_{\text{rel}}, j+j_{\text{rel}}}^n$$



weighted sum of grid values
compact neighborhood
same pattern $\forall$ gridpoints

“ “ A *stencil computation* is an algorithmic motif where ” ”
the value at a certain grid point is computed from a
compact neighborhood of gridpoints in it's vicinity
on the computational grid using the same pattern
for every gridpoint.

Stencil program

$$\frac{\partial \phi}{\partial t} = -\alpha_4 \Delta_h (\Delta_h \phi)$$

Discretization

$$\Delta_h \phi_{i,j}^n \approx (-4 \phi_{i,j}^n + \phi_{i-1,j}^n + \phi_{i+1,j}^n + \phi_{i,j-1}^n + \phi_{i,j+1}^n) / \Delta x^2$$

$$\partial_t \phi_{i,j}^n \approx (\phi_{i,j}^{n+1} - \phi_{i,j}^n) / \Delta t$$

$$\alpha = \alpha_4 \frac{\Delta t}{\Delta x^2}$$

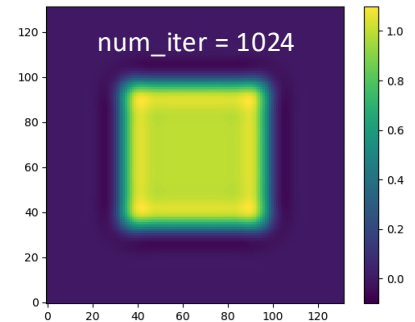
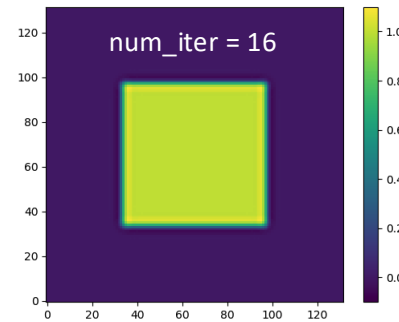
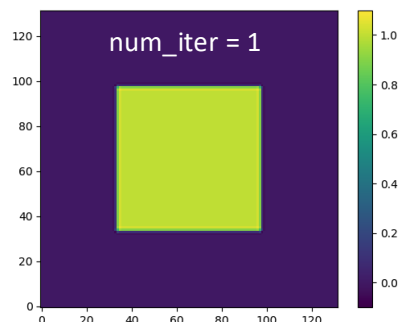
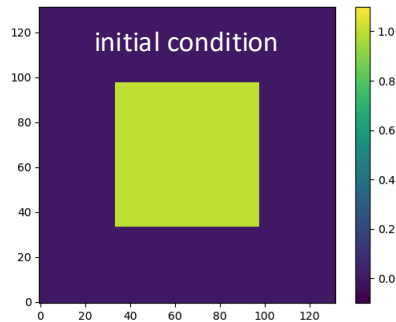
Implementation

```
do num_iter:  
    tmp = lap(in)  
    out = lap(tmp)  
    out = in - alpha * out  
    swap(in, out)
```

Structure of stencil_2d.F90 / stencil_2d.py

$$\text{tmp}_{i,j,k} = \text{lap}(\text{in}_{i,j,k}) = -4 \text{in}_{i,j,k} + \text{in}_{i-1,j,k} + \text{in}_{i+1,j,k} + \text{in}_{i,j-1,k} + \text{in}_{i,j+1,k}$$
$$\text{out}_{i,j,k} = \text{lap}(\text{tmp}_{i,j,k}) = -4 \text{tmp}_{i,j,k} + \text{tmp}_{i-1,j,k} + \text{tmp}_{i+1,j,k} + \text{tmp}_{i,j-1,k} + \text{tmp}_{i,j+1,k}$$
$$\text{out}_{i,j,k} = \text{in}_{i,j,k} - \alpha \text{out}_{i,j,k}$$

`if iter < num_iter : swap(in, out)`



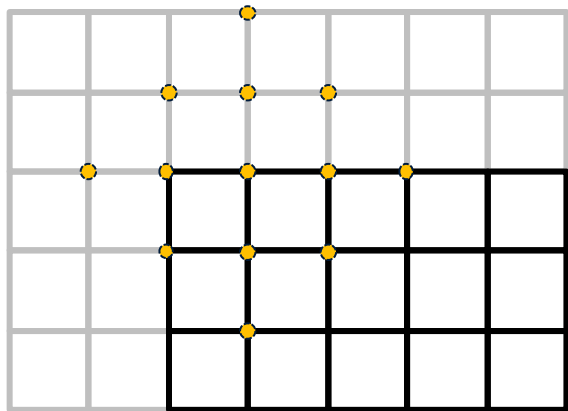
Halo points

`halo_update( in )`

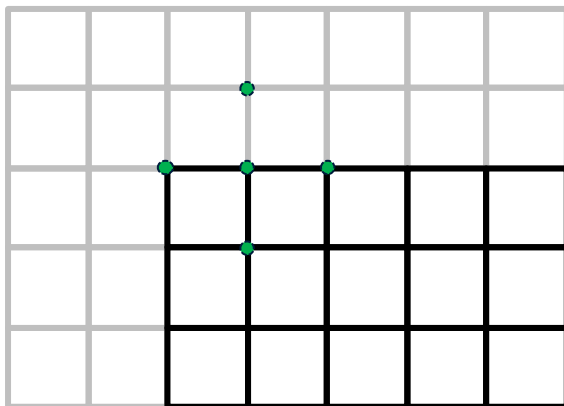
$$\text{tmp}_{i,j,k} = \text{lap}(\text{in}_{i,j,k}) = -4 \text{in}_{i,j,k} + \text{in}_{i-1,j,k} + \text{in}_{i+1,j,k} + \text{in}_{i,j-1,k} + \text{in}_{i,j+1,k}$$

$$\text{out}_{i,j,k} = \text{lap}(\text{tmp}_{i,j,k}) = -4 \text{tmp}_{i,j,k} + \text{tmp}_{i-1,j,k} + \text{tmp}_{i+1,j,k} + \text{tmp}_{i,j-1,k} + \text{tmp}_{i,j+1,k}$$

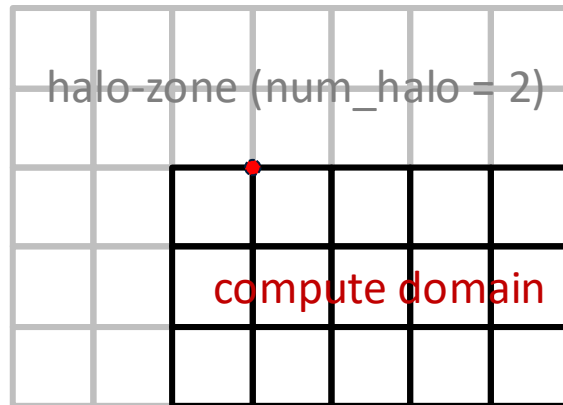
$$\text{out}_{i,j,k} = \text{in}_{i,j,k} - \alpha \text{out}_{i,j,k}$$



`in(i,j,1)`

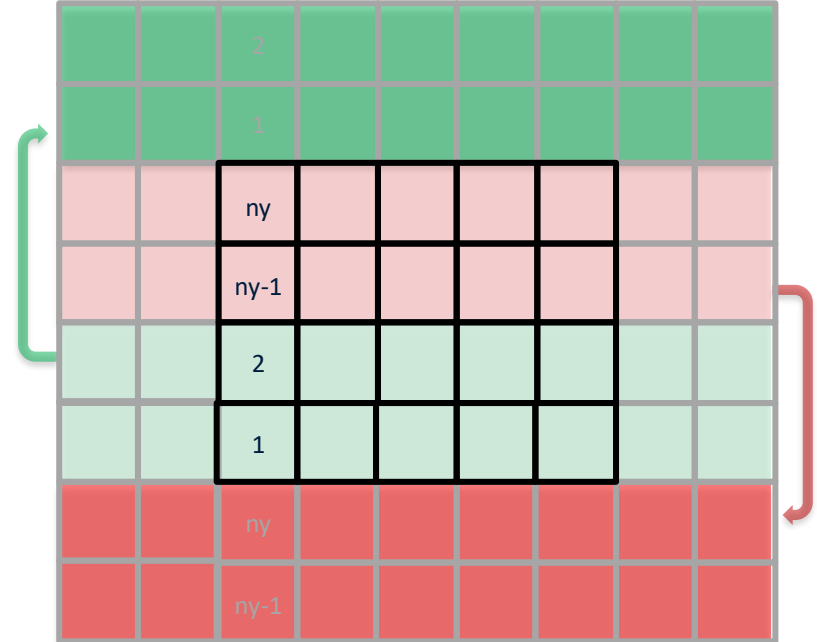
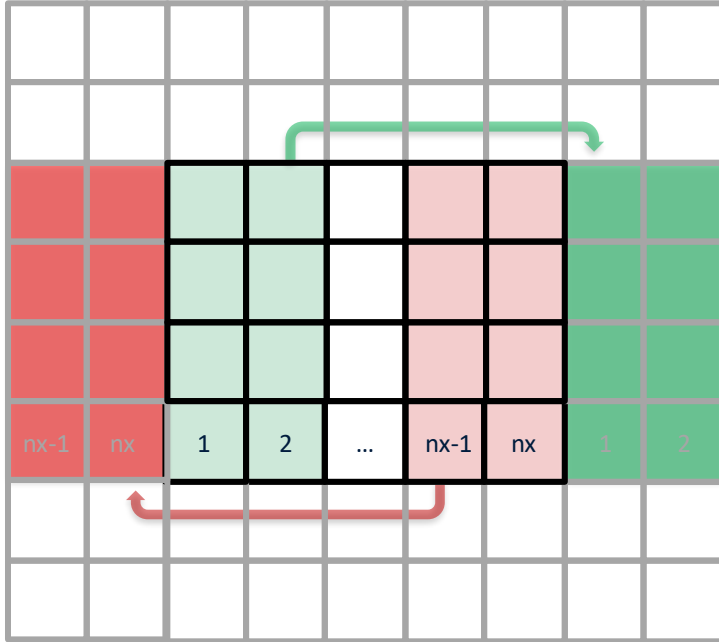


`tmp(i,j,1)`



`out(i,j,1)`

Halo update (periodic)



`field(nx + 2 * num_halo, ny + 2 * num_halo, nz)`

Not stencils

- (Some) implicit methods (e.g. vertical advection)
- Spectral methods (e.g. IFS dynamical core)
- Other algorithmic motifs
 - Reductions
 - Searches
 - ...

IFS vs. AIFS

	IFS	AIFS	Factor
Horizontal resolution (Δx)	TCo1279 / O1280 (~9 km)	N320 (~31 km)	$3.4 \times 3.4 = 11.6$
Vertical resolution (#levels)	137 model levels	14 pressure levels	9.8
Prognostic variables	9 core 3D variables	5 core 3D variables	1.8
Timestep (Δt)	450 s (with substepping)	6 h	48
Total			~10'000

Based purely on state-vector size and forecast-step length, AIFS has a considerable advantage over IFS.

In reality, AIFS is approx. 10-25x faster but with ~1000x less energy/hardware.

Summary

- Spatial resolution drives required computational power $1/\Delta x^3$, this is why we need supercomputers for modeling weather and climate.
- While models use a myriad of different numerical techniques to solve the governing PDEs, the main algorithmic motif resulting from these methods are stencil computations.

Our course will **focus exclusively on stencil computations**, but the concepts are more general and transferable.

Lab Exercises

01-roofline-model.ipynb

- Learn about performance metrics and how to compute theoretical peak values.
- Learn about arithmetic intensity and performance limiters.

02-stencil-program.ipynb

- Determine arithmetic intensity of a stencil program.
- Apply a performance profiling tool to gain insight into performance.
- Show limitations of the von Neumann model for understanding performance.

03-caches-data-locality.ipynb

- Determine arithmetic intensity of a stencil program.
- Apply a performance profiling tool to gain insight into performance.
- Show limitations of the von Neumann model for understanding performance.